LLM Gateway — Provider Abstraction, Adapters & Model Registry

Interview preparation · V1 · ABC interface, adapter pattern, Gemini + Groq, registry

1. The Provider Abstraction

The design pattern at the heart of the gateway.

Q1. How does your provider abstraction work?

MEDIUM

Answer: There is an abstract base class, LLMProvider, with one method: async generate(request) that returns a standardized LLMResponse. Each provider (Gemini, Groq) is a concrete subclass implementing that method. The rest of the app depends only on the abstract type, never on a concrete provider.

Simple explanation: One common interface; each provider is a plug that fits the same socket.

```
class LLMProvider(ABC):
    @abstractmethod
    async def generate(self, request: LLMRequest) -> LLMResponse:
        ...

class GeminiProvider(LLMProvider): ...
class GroqProvider(LLMProvider): ...
```

Interview tip: Name the three pattern words: abstraction, adapter, polymorphism.

Q2. What design patterns are you using here, in plain terms?

MEDIUM

Answer: Adapter pattern: each provider class adapts a foreign API to our common interface. Polymorphism: the gateway calls generate() without knowing which concrete class runs. Dependency inversion: high-level code depends on the LLMProvider abstraction, not on GeminiProvider/GroqProvider.

Simple explanation: Adapters translate; polymorphism lets them be interchangeable; the gateway points at the interface.

Project example: An adapter's job is always: take our LLMRequest, convert to the provider's format, call it, convert the reply back to our LLMResponse.

Likely follow-ups:

- What is an ABC in Python? (a class that can't be instantiated and forces subclasses to implement abstract methods)

Q3. Why keep provider-specific logic out of GatewayService?

HARD

Answer: So the gateway never grows if/elif branches per provider. GatewayService only resolves the model, asks a factory for the matching adapter, and calls generate(). All provider quirks (auth header, field names, response shape) stay inside the adapter.

Simple explanation: No 'if provider == gemini' anywhere in the business layer.

Project example: You can grep the services and API layers for 'if provider ==' and find nothing. Adding Groq in Step 7 required zero changes to GatewayService.

Interview tip: This is the project's strongest architecture point — say it with confidence.

2. Adapters in Detail

How Gemini and Groq differ, and how the adapters hide it.

Q4. How do the Gemini and Groq adapters actually differ?

MEDIUM

Answer: Groq is OpenAI-compatible: roles system/user/assistant pass straight through, the key goes in an Authorization: Bearer header, text is at choices[0].message.content, and usage uses prompt_tokens/completion_tokens. Gemini differs: the key is an x-goog-api-key header, system prompts go in a separate field, the reply is under candidates[].content.parts[].text, and it is a 'thinking' model with its own token accounting.

Simple explanation: Two very different request/response shapes; both normalized to the same internal form.

Project example: Both adapters end by returning the same LLMResponse(content, input_tokens, output_tokens, finish_reason), so callers can't tell them apart.

Likely follow-ups:

- What is response normalization? (mapping each provider's native shape into one common response)

Q5. How would you add a new provider (e.g., a real OpenAI adapter)?

EASY

Answer: Create an OpenAIProvider subclass of LLMProvider implementing generate(); register it in the bootstrap when its API key is present; add its model mapping in the registry (the 'gpt' alias already exists). No changes to routes, GatewayService, or schemas.

Simple explanation: New file + one registration line + one registry entry.

Project example: The registry already maps 'gpt' -> openai and 'claude' -> anthropic; only the adapters are missing, so those currently return 503 PROVIDER_UNAVAILABLE.

Interview tip: This 'add a provider cheaply' story is exactly what interviewers want to hear.

Q6. Why use raw httpx instead of each provider's official SDK?

MEDIUM

Answer: httpx keeps the translation visible and explicit, avoids heavy SDK dependencies and their version churn, and is async-native. Because the adapter pattern isolates provider code, we could swap to an SDK inside one adapter later without affecting anything else.

Simple explanation: One lightweight async HTTP client, full control, easy to reason about.

Project example: Both adapters use httpx.AsyncClient with a shared timeout; the request/response mapping is plain, reviewable code.

Likely follow-ups:

- Trade-off? (we hand-maintain request/response mapping instead of the SDK doing it)

3. The Model Registry

Mapping public names to real provider models.

Q7. What is the model registry and why have one?

EASY

Answer: A single lookup that maps a public alias (e.g., 'gemini', 'groq') to a provider and the real provider-specific model id (e.g., gemini-flash-lite-latest, openai/gpt-oss-20b). It is the one source of truth for 'which model do we serve?'.

Simple explanation: A phonebook: short public name in, provider + real model name out.

Project example: Clients send "model": "gemini"; the registry turns that into provider=google, provider_model=gemini-flash-lite-latest, read from config.

```
ModelConfig('gemini', Provider.GOOGLE, settings.gemini_model)
ModelConfig('groq',   Provider.GROQ,   settings.groq_model)
```

Q8. Why a registry instead of hardcoding model names in the adapters? MEDIUM

Answer: So upgrading a model is a one-value config change, not a code hunt. The real model names come from settings, so GEMINI_MODEL or GROQ_MODEL env vars can override them without touching code. It also gives one place to answer 'is this model known?' and return a clean 404 if not.

Simple explanation: Change the model once in config; everything picks it up.

Project example: When Gemini's default model was slow, switching to gemini-flash-lite-latest was a single registry/config change.

Likely follow-ups:

- What happens on an unknown model? (UnknownModelError -> 404 UNKNOWN_MODEL, before any provider is called)
- What if the provider has no adapter registered? (ProviderUnavailableError -> 503)

Interview tip: Contrast 404 (model not in registry) vs 503 (known model, provider not configured).